

# 计算机组成原理 Project1 实验报告（Week 3）

## 8 位定点卷积-激活-池化链运算器设计

鲁昕宁 2414015

2026 年 4 月 26 日

<https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-CS/tree/master/project1>

# 1 优化模块设计

本部分详细介绍优化模块的设计与验证。

## 1.1 优化模块源码

在基础模块之上，实现了以下优化模块：

模块 mul8\_booth\_wallace 的 Verilog 源码

对应文件：../src/optimized/mul8\_booth\_wallace.v

```
module mul8_booth_wallace (
    input  wire [7:0] a_u8,
    input  wire signed [7:0] b_s8,
    output wire signed [15:0] p_s16
);
    function automatic signed [16:0] booth_select;
        input [2:0] booth_bits;
        input signed [16:0] multiplicand;
        begin
            case (booth_bits)
                3'b000, 3'b111: booth_select = 17'sd0;
                3'b001, 3'b010: booth_select = multiplicand;
                3'b011: booth_select = multiplicand <<< 1;
                3'b100: booth_select = -(multiplicand <<< 1);
                3'b101, 3'b110: booth_select = -multiplicand;
                default: booth_select = 17'sd0;
            endcase
        end
    endfunction

    wire signed [16:0] a_ext;
    wire [2:0] booth0;
    wire [2:0] booth1;
    wire [2:0] booth2;
    wire [2:0] booth3;

    wire signed [16:0] pp0;
    wire signed [16:0] pp1;
    wire signed [16:0] pp2;
    wire signed [16:0] pp3;

    wire [16:0] csa1_sum;
    wire [16:0] csa1_carry;
    wire [16:0] csa2_sum;
```

```

wire [16:0] csa2_carry;
wire signed [16:0] prod_ext;

assign a_ext = $signed({9'd0, a_u8});

assign booth0 = {b_s8[1], b_s8[0], 1'b0};
assign booth1 = {b_s8[3], b_s8[2], b_s8[1]};
assign booth2 = {b_s8[5], b_s8[4], b_s8[3]};
assign booth3 = {b_s8[7], b_s8[6], b_s8[5]};

assign pp0 = booth_select(booth0, a_ext);
assign pp1 = booth_select(booth1, a_ext) <<< 2;
assign pp2 = booth_select(booth2, a_ext) <<< 4;
assign pp3 = booth_select(booth3, a_ext) <<< 6;

assign csa1_sum = pp0 ^ pp1 ^ pp2;
assign csa1_carry = ((pp0 & pp1) | (pp0 & pp2) | (pp1 & pp2)) << 1;

assign csa2_sum = csa1_sum ^ csa1_carry ^ pp3;
assign csa2_carry = ((csa1_sum & csa1_carry) | (csa1_sum & pp3) |
↳ (csa1_carry & pp3)) << 1;

assign prod_ext = $signed(csa2_sum) + $signed(csa2_carry);
assign p_s16 = prod_ext[15:0];
endmodule

```

## 模块 simd\_add8x4 的 Verilog 源码

对应文件: ../src/optimized/simd\_add8x4.v

```

module simd_add8x4 (
    input wire [31:0] a_u8x4,
    input wire [31:0] b_u8x4,
    output wire [35:0] y_u9x4
);
    wire [8:0] s0;
    wire [8:0] s1;
    wire [8:0] s2;
    wire [8:0] s3;

    assign s0 = {1'b0, a_u8x4[7:0]} + {1'b0, b_u8x4[7:0]};
    assign s1 = {1'b0, a_u8x4[15:8]} + {1'b0, b_u8x4[15:8]};
    assign s2 = {1'b0, a_u8x4[23:16]} + {1'b0, b_u8x4[23:16]};
    assign s3 = {1'b0, a_u8x4[31:24]} + {1'b0, b_u8x4[31:24]};

    assign y_u9x4 = {s3, s2, s1, s0};
endmodule

```

## 模块 simd\_mul8x4 的 Verilog 源码

对应文件: ../src/optimized/simd\_mul8x4.v

```
module simd_mul8x4 (  
    input  wire [31:0] a_u8x4,  
    input  wire [31:0] b_s8x4,  
    output wire [63:0] p_s16x4  
);  
    wire signed [7:0] b0_s8;  
    wire signed [7:0] b1_s8;  
    wire signed [7:0] b2_s8;  
    wire signed [7:0] b3_s8;  
  
    wire signed [15:0] p0;  
    wire signed [15:0] p1;  
    wire signed [15:0] p2;  
    wire signed [15:0] p3;  
  
    assign b0_s8 = b_s8x4[7:0];  
    assign b1_s8 = b_s8x4[15:8];  
    assign b2_s8 = b_s8x4[23:16];  
    assign b3_s8 = b_s8x4[31:24];  
  
    mul8_booth_wallace u_lane0 (  
        .a_u8(a_u8x4[7:0]),  
        .b_s8(b0_s8),  
        .p_s16(p0)  
    );  
  
    mul8_booth_wallace u_lane1 (  
        .a_u8(a_u8x4[15:8]),  
        .b_s8(b1_s8),  
        .p_s16(p1)  
    );  
  
    mul8_booth_wallace u_lane2 (  
        .a_u8(a_u8x4[23:16]),  
        .b_s8(b2_s8),  
        .p_s16(p2)  
    );  
  
    mul8_booth_wallace u_lane3 (  
        .a_u8(a_u8x4[31:24]),  
        .b_s8(b3_s8),  
        .p_s16(p3)  
    );  
);
```

```

    assign p_s16x4 = {p3, p2, p1, p0};
endmodule

```

## 模块 simd\_relu8x4 的 Verilog 源码

对应文件: ../src/optimized/simd\_relu8x4.v

```

module simd_relu8x4 (
    input  wire signed [79:0] x_s20x4,
    output wire [31:0] y_u8x4
);
    function [7:0] relu_clip_lane;
        input signed [19:0] x_s20;
        begin
            if (x_s20 < 0) begin
                relu_clip_lane = 8'd0;
            end else if (x_s20 > 20'sd255) begin
                relu_clip_lane = 8'd255;
            end else begin
                relu_clip_lane = x_s20[7:0];
            end
        end
    endfunction

    wire [7:0] y0;
    wire [7:0] y1;
    wire [7:0] y2;
    wire [7:0] y3;

    assign y0 = relu_clip_lane(x_s20x4[19:0]);
    assign y1 = relu_clip_lane(x_s20x4[39:20]);
    assign y2 = relu_clip_lane(x_s20x4[59:40]);
    assign y3 = relu_clip_lane(x_s20x4[79:60]);

    assign y_u8x4 = {y3, y2, y1, y0};
endmodule

```

## 模块 simd\_avgpool2x2\_8bit 的 Verilog 源码

对应文件: ../src/optimized/simd\_avgpool2x2\_8bit.v

```

module simd_avgpool2x2_8bit (
    input  wire [31:0] x_u8x4,
    output wire [7:0] y_u8
);
    wire [35:0] pair_sum_u9x4;

```

```

wire [8:0] pair0_u9;
wire [8:0] pair1_u9;
wire [9:0] pool_sum_u10;

simd_add8x4 u_pair_add (
    .a_u8x4({16'd0, x_u8x4[31:24], x_u8x4[15:8]}),
    .b_u8x4({16'd0, x_u8x4[23:16], x_u8x4[7:0]}),
    .y_u9x4(pair_sum_u9x4)
);

assign pair0_u9 = pair_sum_u9x4[8:0];
assign pair1_u9 = pair_sum_u9x4[17:9];
assign pool_sum_u10 = {1'b0, pair0_u9} + {1'b0, pair1_u9};
assign y_u8 = pool_sum_u10[9:2];
endmodule

```

## 模块 mul8\_dsp48e1 的 Verilog 源码

对应文件: ../src/optimized/mul8\_dsp48e1.v

```

module mul8_dsp48e1 (
    input wire [7:0] a_u8,
    input wire signed [7:0] b_s8,
    output wire signed [15:0] p_s16
);
    wire signed [24:0] a_mul_s25;
    wire signed [17:0] b_mul_s18;
    wire signed [47:0] p_dsp_s48;

    // Map the project operand types directly onto the DSP48E1 multiplier:
    // unsigned 8-bit pixel -> positive signed 25-bit operand
    // signed 8-bit weight -> sign-extended 18-bit operand
    assign a_mul_s25 = {17'd0, a_u8};
    assign b_mul_s18 = {{10{b_s8[7]}}, b_s8};

    DSP48E1 #(
        .ACASCREG(0),
        .ADREG(0),
        .ALUMODEREG(0),
        .AREG(0),
        .A_INPUT("DIRECT"),
        .BCASCREG(0),
        .BREG(0),
        .B_INPUT("DIRECT"),
        .CARRYINREG(0),
        .CARRYINSELREG(0),
        .CREG(0),

```

```

.DREG(0),
.INMODEREG(0),
.MREG(0),
.OPMODEREG(0),
.PREG(0),
.USE_DPORT("FALSE"),
.USE_MULT("MULTIPLY"),
.USE_SIMD("ONE48")
) u_dsp48e1 (
.ACOUT(),
.BCOUT(),
.CARRYCASCOUT(),
.CARRYOUT(),
.MULTSIGNOUT(),
.OVERFLOW(),
.P(p_dsp_s48),
.PATTERNBDETECT(),
.PATTERNDETECT(),
.PCOUT(),
.UNDERFLOW(),
.A({5'd0, a_mul_s25}),
.ACIN(30'd0),
.ALUMODE(4'b0000),
.B(b_mul_s18),
.BCIN(18'd0),
.C(48'd0),
.CARRYCASCIN(1'b0),
.CARRYIN(1'b0),
.CARRYINSEL(3'b000),
.CEA1(1'b0),
.CEA2(1'b0),
.CEAD(1'b0),
.CEALUMODE(1'b0),
.CEB1(1'b0),
.CEB2(1'b0),
.CEC(1'b0),
.CECARRYIN(1'b0),
.CECTRL(1'b0),
.CED(1'b0),
.CEINMODE(1'b0),
.CEM(1'b0),
.CEP(1'b0),
.CLK(1'b0),
.D(25'd0),
.INMODE(5'b00000),
.MULTSIGNIN(1'b0),

```

```

        .OPMODE(7'b0000101),
        .PCIN(48'd0),
        .RSTA(1'b0),
        .STALLCARRYIN(1'b0),
        .RSTALUMODE(1'b0),
        .RSTB(1'b0),
        .RSTC(1'b0),
        .RSTCTRL(1'b0),
        .RSTD(1'b0),
        .RSTINMODE(1'b0),
        .RSTM(1'b0),
        .RSTP(1'b0)
    );

    assign p_s16 = p_dsp_s48[15:0];
endmodule

```

### 模块 simd\_mul8x4\_dsp 的 Verilog 源码

对应文件: ../src/optimized/simd\_mul8x4\_dsp.v

```

module simd_mul8x4_dsp (
    input  wire [31:0] a_u8x4,
    input  wire [31:0] b_s8x4,
    output wire [63:0] p_s16x4
);

    wire signed [7:0] b0_s8;
    wire signed [7:0] b1_s8;
    wire signed [7:0] b2_s8;
    wire signed [7:0] b3_s8;

    wire signed [15:0] p0;
    wire signed [15:0] p1;
    wire signed [15:0] p2;
    wire signed [15:0] p3;

    assign b0_s8 = b_s8x4[7:0];
    assign b1_s8 = b_s8x4[15:8];
    assign b2_s8 = b_s8x4[23:16];
    assign b3_s8 = b_s8x4[31:24];

    mul8_dsp48e1 u_lane0 (
        .a_u8(a_u8x4[7:0]),
        .b_s8(b0_s8),
        .p_s16(p0)
    );

```



```

mul8_dsp48e1 u_lane1 (
    .a_u8(a_u8x4[15:8]),
    .b_s8(b1_s8),
    .p_s16(p1)
);

mul8_dsp48e1 u_lane2 (
    .a_u8(a_u8x4[23:16]),
    .b_s8(b2_s8),
    .p_s16(p2)
);

mul8_dsp48e1 u_lane3 (
    .a_u8(a_u8x4[31:24]),
    .b_s8(b3_s8),
    .p_s16(p3)
);

assign p_s16x4 = {p3, p2, p1, p0};
endmodule

```

优化版本顶层模块源码如下：

### 顶层 cnn\_compare\_booth\_top 的 Verilog 源码

对应文件： ../src/optimized/cnn\_compare\_booth\_top.v

```

module cnn_compare_booth_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_base u_booth_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .use_booth(1'b1),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule

```

## 顶层 cnn\_compare\_booth\_simd\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_top.v

```
module cnn_compare_booth_simd_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd u_booth_simd (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 顶层 cnn\_compare\_booth\_simd\_pipe\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_pipe\_top.v

```
module cnn_compare_booth_simd_pipe_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd_pipe u_booth_simd_pipe (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 顶层 cnn\_compare\_booth\_simd\_pipe\_dsp\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_pipe\_dsp\_top.v

```
module cnn_compare_booth_simd_pipe_dsp_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd_pipe_dsp u_booth_simd_pipe_dsp (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 模块 cnn\_chain\_core\_booth\_simd 的 Verilog 源码

对应文件: ../src/optimized/cnn\_chain\_core\_booth\_simd.v

```
module cnn_chain_core_booth_simd (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_u8x4;
    wire [31:0] relu_blk1_u8x4;
    wire [31:0] relu_blk2_u8x4;

    wire [7:0] relu0;
    wire [7:0] relu1;
```

```

wire [7:0] relu2;
wire [7:0] relu3;
wire [7:0] relu4;
wire [7:0] relu5;
wire [7:0] relu6;
wire [7:0] relu7;
wire [7:0] relu8;

wire [7:0] pool0_u8;
wire [7:0] pool1_u8;
wire [7:0] pool2_u8;
wire [7:0] pool3_u8;
wire [31:0] out_comb_u8x4;

cnn_conv3x3_simd_core #(
    .USE_DSP(1'b0)
) u_conv (
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .conv_blk0_s20x4(conv_blk0_s20x4),
    .conv_blk1_s20x4(conv_blk1_s20x4),
    .conv_blk2_s20x4(conv_blk2_s20x4)
);

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_s20x4),
    .y_u8x4(relu_blk0_u8x4)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_s20x4),
    .y_u8x4(relu_blk1_u8x4)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_s20x4),
    .y_u8x4(relu_blk2_u8x4)
);

assign relu0 = relu_blk0_u8x4[7:0];
assign relu1 = relu_blk0_u8x4[15:8];
assign relu2 = relu_blk0_u8x4[23:16];
assign relu3 = relu_blk0_u8x4[31:24];
assign relu4 = relu_blk1_u8x4[7:0];
assign relu5 = relu_blk1_u8x4[15:8];
assign relu6 = relu_blk1_u8x4[23:16];

```

```

assign relu7 = relu_blk1_u8x4[31:24];
assign relu8 = relu_blk2_u8x4[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

assign out_comb_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        done <= 1'b0;
        out_flat_u8x4 <= 32'd0;
    end else begin
        done <= start;
        if (start) begin
            out_flat_u8x4 <= out_comb_u8x4;
        end
    end
end

endmodule

```

### 模块 cnn\_chain\_core\_booth\_simd\_pipe 的 Verilog 源码

对应文件: ../src/optimized/cnn\_chain\_core\_booth\_simd\_pipe.v

```

module cnn_chain_core_booth_simd_pipe (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,

```

```

    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    reg [2:0] vld_pipe;

    reg signed [79:0] conv_blk0_reg;
    reg signed [79:0] conv_blk1_reg;
    reg signed [79:0] conv_blk2_reg;

    reg [31:0] relu_blk0_reg;
    reg [31:0] relu_blk1_reg;
    reg [31:0] relu_blk2_reg;

    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_stage2;
    wire [31:0] relu_blk1_stage2;
    wire [31:0] relu_blk2_stage2;

    wire [7:0] relu0;
    wire [7:0] relu1;
    wire [7:0] relu2;
    wire [7:0] relu3;
    wire [7:0] relu4;
    wire [7:0] relu5;
    wire [7:0] relu6;
    wire [7:0] relu7;
    wire [7:0] relu8;

    wire [7:0] pool0_u8;
    wire [7:0] pool1_u8;
    wire [7:0] pool2_u8;
    wire [7:0] pool3_u8;
    wire [31:0] out_stage3_u8x4;

    cnn_conv3x3_simd_core #(
        .USE_DSP(1'b0)
    ) u_conv (
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .conv_blk0_s20x4(conv_blk0_s20x4),
        .conv_blk1_s20x4(conv_blk1_s20x4),
        .conv_blk2_s20x4(conv_blk2_s20x4)

```

```

);

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_reg),
    .y_u8x4(relu_blk0_stage2)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_reg),
    .y_u8x4(relu_blk1_stage2)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_reg),
    .y_u8x4(relu_blk2_stage2)
);

assign relu0 = relu_blk0_reg[7:0];
assign relu1 = relu_blk0_reg[15:8];
assign relu2 = relu_blk0_reg[23:16];
assign relu3 = relu_blk0_reg[31:24];
assign relu4 = relu_blk1_reg[7:0];
assign relu5 = relu_blk1_reg[15:8];
assign relu6 = relu_blk1_reg[23:16];
assign relu7 = relu_blk1_reg[31:24];
assign relu8 = relu_blk2_reg[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

```

```

assign out_stage3_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        vld_pipe <= 3'b000;
        conv_blk0_reg <= 80'sd0;
        conv_blk1_reg <= 80'sd0;
        conv_blk2_reg <= 80'sd0;
        relu_blk0_reg <= 32'd0;
        relu_blk1_reg <= 32'd0;
        relu_blk2_reg <= 32'd0;
        out_flat_u8x4 <= 32'd0;
        done <= 1'b0;
    end else begin
        vld_pipe <= {vld_pipe[1:0], start};
        done <= vld_pipe[1];

        if (start) begin
            conv_blk0_reg <= conv_blk0_s20x4;
            conv_blk1_reg <= conv_blk1_s20x4;
            conv_blk2_reg <= conv_blk2_s20x4;
        end

        if (vld_pipe[0]) begin
            relu_blk0_reg <= relu_blk0_stage2;
            relu_blk1_reg <= relu_blk1_stage2;
            relu_blk2_reg <= relu_blk2_stage2;
        end

        if (vld_pipe[1]) begin
            out_flat_u8x4 <= out_stage3_u8x4;
        end
    end
end
endmodule

```

模块 `cnn_chain_core_booth_simd_pipe_dsp` 的 Verilog 源码

对应文件: `../src/optimized/cnn_chain_core_booth_simd_pipe_dsp.v`

```

module cnn_chain_core_booth_simd_pipe_dsp (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,

```



```

    output reg  done,
    output reg  [31:0] out_flat_u8x4
);

    reg [2:0] vld_pipe;

    reg signed [79:0] conv_blk0_reg;
    reg signed [79:0] conv_blk1_reg;
    reg signed [79:0] conv_blk2_reg;

    reg [31:0] relu_blk0_reg;
    reg [31:0] relu_blk1_reg;
    reg [31:0] relu_blk2_reg;

    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_stage2;
    wire [31:0] relu_blk1_stage2;
    wire [31:0] relu_blk2_stage2;

    wire [7:0] relu0;
    wire [7:0] relu1;
    wire [7:0] relu2;
    wire [7:0] relu3;
    wire [7:0] relu4;
    wire [7:0] relu5;
    wire [7:0] relu6;
    wire [7:0] relu7;
    wire [7:0] relu8;

    wire [7:0] pool0_u8;
    wire [7:0] pool1_u8;
    wire [7:0] pool2_u8;
    wire [7:0] pool3_u8;
    wire [31:0] out_stage3_u8x4;

    cnn_conv3x3_simd_core #(
        .USE_DSP(1'b1)
    ) u_conv (
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .conv_blk0_s20x4(conv_blk0_s20x4),
        .conv_blk1_s20x4(conv_blk1_s20x4),
        .conv_blk2_s20x4(conv_blk2_s20x4)
    );

```

```

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_reg),
    .y_u8x4(relu_blk0_stage2)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_reg),
    .y_u8x4(relu_blk1_stage2)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_reg),
    .y_u8x4(relu_blk2_stage2)
);

assign relu0 = relu_blk0_reg[7:0];
assign relu1 = relu_blk0_reg[15:8];
assign relu2 = relu_blk0_reg[23:16];
assign relu3 = relu_blk0_reg[31:24];
assign relu4 = relu_blk1_reg[7:0];
assign relu5 = relu_blk1_reg[15:8];
assign relu6 = relu_blk1_reg[23:16];
assign relu7 = relu_blk1_reg[31:24];
assign relu8 = relu_blk2_reg[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

```

```

assign out_stage3_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        vld_pipe <= 3'b000;
        conv_blk0_reg <= 80'sd0;
        conv_blk1_reg <= 80'sd0;
        conv_blk2_reg <= 80'sd0;
        relu_blk0_reg <= 32'd0;
        relu_blk1_reg <= 32'd0;
        relu_blk2_reg <= 32'd0;
        out_flat_u8x4 <= 32'd0;
        done <= 1'b0;
    end else begin
        vld_pipe <= {vld_pipe[1:0], start};
        done <= vld_pipe[1];

        if (start) begin
            conv_blk0_reg <= conv_blk0_s20x4;
            conv_blk1_reg <= conv_blk1_s20x4;
            conv_blk2_reg <= conv_blk2_s20x4;
        end

        if (vld_pipe[0]) begin
            relu_blk0_reg <= relu_blk0_stage2;
            relu_blk1_reg <= relu_blk1_stage2;
            relu_blk2_reg <= relu_blk2_stage2;
        end

        if (vld_pipe[1]) begin
            out_flat_u8x4 <= out_stage3_u8x4;
        end
    end
end
endmodule

```

### 共享卷积核心 cnn\_conv3x3\_simd\_core 的 Verilog 源码

对应文件: ../src/optimized/cnn\_conv3x3\_simd\_core.v

```

module cnn_conv3x3_simd_core #(
    parameter USE_DSP = 1'b0
) (
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire signed [79:0] conv_blk0_s20x4,
    output wire signed [79:0] conv_blk1_s20x4,

```

```

output wire signed [79:0] conv_blk2_s20x4
);
wire [31:0] pix_blk0 [0:8];
wire [31:0] pix_blk1 [0:8];
wire [31:0] pix_blk2 [0:8];
wire [31:0] ker_pack [0:8];

wire [63:0] prod_blk0 [0:8];
wire [63:0] prod_blk1 [0:8];
wire [63:0] prod_blk2 [0:8];

wire signed [19:0] conv0;
wire signed [19:0] conv1;
wire signed [19:0] conv2;
wire signed [19:0] conv3;
wire signed [19:0] conv4;
wire signed [19:0] conv5;
wire signed [19:0] conv6;
wire signed [19:0] conv7;
wire signed [19:0] conv8;

function signed [19:0] lane20;
    input [63:0] lane_bus;
    input integer lane_idx;
    reg signed [15:0] lane16;
    begin
        case (lane_idx)
            0: lane16 = lane_bus[15:0];
            1: lane16 = lane_bus[31:16];
            2: lane16 = lane_bus[47:32];
            default: lane16 = lane_bus[63:48];
        endcase
        lane20 = {{4{lane16[15]}}, lane16};
    end
endfunction

assign ker_pack[0] = {4{ker_flat_s8x9[7:0]}};
assign ker_pack[1] = {4{ker_flat_s8x9[15:8]}};
assign ker_pack[2] = {4{ker_flat_s8x9[23:16]}};
assign ker_pack[3] = {4{ker_flat_s8x9[31:24]}};
assign ker_pack[4] = {4{ker_flat_s8x9[39:32]}};
assign ker_pack[5] = {4{ker_flat_s8x9[47:40]}};
assign ker_pack[6] = {4{ker_flat_s8x9[55:48]}};
assign ker_pack[7] = {4{ker_flat_s8x9[63:56]}};
assign ker_pack[8] = {4{ker_flat_s8x9[71:64]}};

```

```

assign pix_blk0[0] = {img_flat_u8x25[47:40], img_flat_u8x25[23:16],
  ↪ img_flat_u8x25[15:8], img_flat_u8x25[7:0]};
assign pix_blk0[1] = {img_flat_u8x25[55:48], img_flat_u8x25[31:24],
  ↪ img_flat_u8x25[23:16], img_flat_u8x25[15:8]};
assign pix_blk0[2] = {img_flat_u8x25[63:56], img_flat_u8x25[39:32],
  ↪ img_flat_u8x25[31:24], img_flat_u8x25[23:16]};
assign pix_blk0[3] = {img_flat_u8x25[87:80], img_flat_u8x25[63:56],
  ↪ img_flat_u8x25[55:48], img_flat_u8x25[47:40]};
assign pix_blk0[4] = {img_flat_u8x25[95:88], img_flat_u8x25[71:64],
  ↪ img_flat_u8x25[63:56], img_flat_u8x25[55:48]};
assign pix_blk0[5] = {img_flat_u8x25[103:96], img_flat_u8x25[79:72],
  ↪ img_flat_u8x25[71:64], img_flat_u8x25[63:56]};
assign pix_blk0[6] = {img_flat_u8x25[127:120],img_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88], img_flat_u8x25[87:80]};
assign pix_blk0[7] = {img_flat_u8x25[135:128],img_flat_u8x25[111:104],i_j
  ↪ mg_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88]};
assign pix_blk0[8] = {img_flat_u8x25[143:136],img_flat_u8x25[119:112],i_j
  ↪ mg_flat_u8x25[111:104],img_flat_u8x25[103:96]};

assign pix_blk1[0] = {img_flat_u8x25[95:88], img_flat_u8x25[87:80],
  ↪ img_flat_u8x25[63:56], img_flat_u8x25[55:48]};
assign pix_blk1[1] = {img_flat_u8x25[103:96], img_flat_u8x25[95:88],
  ↪ img_flat_u8x25[71:64], img_flat_u8x25[63:56]};
assign pix_blk1[2] = {img_flat_u8x25[111:104],img_flat_u8x25[103:96],
  ↪ img_flat_u8x25[79:72], img_flat_u8x25[71:64]};
assign pix_blk1[3] = {img_flat_u8x25[135:128],img_flat_u8x25[127:120],i_j
  ↪ mg_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88]};
assign pix_blk1[4] = {img_flat_u8x25[143:136],img_flat_u8x25[135:128],i_j
  ↪ mg_flat_u8x25[111:104],img_flat_u8x25[103:96]};
assign pix_blk1[5] = {img_flat_u8x25[151:144],img_flat_u8x25[143:136],i_j
  ↪ mg_flat_u8x25[119:112],img_flat_u8x25[111:104]};
assign pix_blk1[6] = {img_flat_u8x25[175:168],img_flat_u8x25[167:160],i_j
  ↪ mg_flat_u8x25[143:136],img_flat_u8x25[135:128]};
assign pix_blk1[7] = {img_flat_u8x25[183:176],img_flat_u8x25[175:168],i_j
  ↪ mg_flat_u8x25[151:144],img_flat_u8x25[143:136]};
assign pix_blk1[8] = {img_flat_u8x25[191:184],img_flat_u8x25[183:176],i_j
  ↪ mg_flat_u8x25[159:152],img_flat_u8x25[151:144]};

assign pix_blk2[0] = {24'd0, img_flat_u8x25[103:96]};
assign pix_blk2[1] = {24'd0, img_flat_u8x25[111:104]};
assign pix_blk2[2] = {24'd0, img_flat_u8x25[119:112]};
assign pix_blk2[3] = {24'd0, img_flat_u8x25[143:136]};
assign pix_blk2[4] = {24'd0, img_flat_u8x25[151:144]};
assign pix_blk2[5] = {24'd0, img_flat_u8x25[159:152]};

```

```

assign pix_blk2[6] = {24'd0, img_flat_u8x25[183:176]};
assign pix_blk2[7] = {24'd0, img_flat_u8x25[191:184]};
assign pix_blk2[8] = {24'd0, img_flat_u8x25[199:192]};

genvar gk0;
generate
    for (gk0 = 0; gk0 < 9; gk0 = gk0 + 1) begin : gen_blk0_mul
        if (USE_DSP) begin : gen_blk0_dsp
            simd_mul8x4_dsp u_mul (
                .a_u8x4(pix_blk0[gk0]),
                .b_s8x4(ker_pack[gk0]),
                .p_s16x4(prod_blk0[gk0])
            );
        end else begin : gen_blk0_booth
            simd_mul8x4 u_mul (
                .a_u8x4(pix_blk0[gk0]),
                .b_s8x4(ker_pack[gk0]),
                .p_s16x4(prod_blk0[gk0])
            );
        end
    end
endgenerate

genvar gk1;
generate
    for (gk1 = 0; gk1 < 9; gk1 = gk1 + 1) begin : gen_blk1_mul
        if (USE_DSP) begin : gen_blk1_dsp
            simd_mul8x4_dsp u_mul (
                .a_u8x4(pix_blk1[gk1]),
                .b_s8x4(ker_pack[gk1]),
                .p_s16x4(prod_blk1[gk1])
            );
        end else begin : gen_blk1_booth
            simd_mul8x4 u_mul (
                .a_u8x4(pix_blk1[gk1]),
                .b_s8x4(ker_pack[gk1]),
                .p_s16x4(prod_blk1[gk1])
            );
        end
    end
endgenerate

genvar gk2;
generate
    for (gk2 = 0; gk2 < 9; gk2 = gk2 + 1) begin : gen_blk2_mul
        if (USE_DSP) begin : gen_blk2_dsp

```

```

        simd_mul8x4_dsp u_mul (
            .a_u8x4(pix_blk2[gk2]),
            .b_s8x4(ker_pack[gk2]),
            .p_s16x4(prod_blk2[gk2])
        );
    end else begin : gen_blk2_booth
        simd_mul8x4 u_mul (
            .a_u8x4(pix_blk2[gk2]),
            .b_s8x4(ker_pack[gk2]),
            .p_s16x4(prod_blk2[gk2])
        );
    end
end
endgenerate

assign conv0 = lane20(prod_blk0[0], 0) + lane20(prod_blk0[1], 0) +
↳ lane20(prod_blk0[2], 0) +
    lane20(prod_blk0[3], 0) + lane20(prod_blk0[4], 0) +
    ↳ lane20(prod_blk0[5], 0) +
    lane20(prod_blk0[6], 0) + lane20(prod_blk0[7], 0) +
    ↳ lane20(prod_blk0[8], 0);
assign conv1 = lane20(prod_blk0[0], 1) + lane20(prod_blk0[1], 1) +
↳ lane20(prod_blk0[2], 1) +
    lane20(prod_blk0[3], 1) + lane20(prod_blk0[4], 1) +
    ↳ lane20(prod_blk0[5], 1) +
    lane20(prod_blk0[6], 1) + lane20(prod_blk0[7], 1) +
    ↳ lane20(prod_blk0[8], 1);
assign conv2 = lane20(prod_blk0[0], 2) + lane20(prod_blk0[1], 2) +
↳ lane20(prod_blk0[2], 2) +
    lane20(prod_blk0[3], 2) + lane20(prod_blk0[4], 2) +
    ↳ lane20(prod_blk0[5], 2) +
    lane20(prod_blk0[6], 2) + lane20(prod_blk0[7], 2) +
    ↳ lane20(prod_blk0[8], 2);
assign conv3 = lane20(prod_blk0[0], 3) + lane20(prod_blk0[1], 3) +
↳ lane20(prod_blk0[2], 3) +
    lane20(prod_blk0[3], 3) + lane20(prod_blk0[4], 3) +
    ↳ lane20(prod_blk0[5], 3) +
    lane20(prod_blk0[6], 3) + lane20(prod_blk0[7], 3) +
    ↳ lane20(prod_blk0[8], 3);

assign conv4 = lane20(prod_blk1[0], 0) + lane20(prod_blk1[1], 0) +
↳ lane20(prod_blk1[2], 0) +
    lane20(prod_blk1[3], 0) + lane20(prod_blk1[4], 0) +
    ↳ lane20(prod_blk1[5], 0) +
    lane20(prod_blk1[6], 0) + lane20(prod_blk1[7], 0) +
    ↳ lane20(prod_blk1[8], 0);

```

```

assign conv5 = lane20(prod_blk1[0], 1) + lane20(prod_blk1[1], 1) +
↳ lane20(prod_blk1[2], 1) +
    lane20(prod_blk1[3], 1) + lane20(prod_blk1[4], 1) +
    ↳ lane20(prod_blk1[5], 1) +
    lane20(prod_blk1[6], 1) + lane20(prod_blk1[7], 1) +
    ↳ lane20(prod_blk1[8], 1);
assign conv6 = lane20(prod_blk1[0], 2) + lane20(prod_blk1[1], 2) +
↳ lane20(prod_blk1[2], 2) +
    lane20(prod_blk1[3], 2) + lane20(prod_blk1[4], 2) +
    ↳ lane20(prod_blk1[5], 2) +
    lane20(prod_blk1[6], 2) + lane20(prod_blk1[7], 2) +
    ↳ lane20(prod_blk1[8], 2);
assign conv7 = lane20(prod_blk1[0], 3) + lane20(prod_blk1[1], 3) +
↳ lane20(prod_blk1[2], 3) +
    lane20(prod_blk1[3], 3) + lane20(prod_blk1[4], 3) +
    ↳ lane20(prod_blk1[5], 3) +
    lane20(prod_blk1[6], 3) + lane20(prod_blk1[7], 3) +
    ↳ lane20(prod_blk1[8], 3);

assign conv8 = lane20(prod_blk2[0], 0) + lane20(prod_blk2[1], 0) +
↳ lane20(prod_blk2[2], 0) +
    lane20(prod_blk2[3], 0) + lane20(prod_blk2[4], 0) +
    ↳ lane20(prod_blk2[5], 0) +
    lane20(prod_blk2[6], 0) + lane20(prod_blk2[7], 0) +
    ↳ lane20(prod_blk2[8], 0);

assign conv_blk0_s20x4 = {conv3, conv2, conv1, conv0};
assign conv_blk1_s20x4 = {conv7, conv6, conv5, conv4};
assign conv_blk2_s20x4 = {20'sd0, 20'sd0, 20'sd0, conv8};
endmodule

```

## 1.2 优化模块仿真

优化模块的 testbench 覆盖范围如表 1 所示。

表 1: 优化模块 testbench

| testbench           | 验证对象             | 覆盖方式                    |
|---------------------|------------------|-------------------------|
| tb_mul_compare      | BoothWallace 乘法器 | 与 mul8 做 65536 组穷举一致性对比 |
| tb_simd_mul_compare | SIMD / DSP 乘法器   | lane 等价性、边界值与混合模式对比     |

Vivado 日志显示上述 testbench 均通过：

- mul8 vs booth/wallace: PASS
- tb\_simd\_mul\_compare PASS



## testbench tb\_mul\_compare 的 Verilog 源码

对应文件: ../sim/tb\_mul\_compare.v

```
`timescale 1ns/1ps

module tb_mul_compare;
    reg [7:0] a_u8;
    reg signed [7:0] b_s8;
    wire signed [15:0] p_base;
    wire signed [15:0] p_booth;
    integer ia;
    integer ib;

    mul8 u_base (
        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_base)
    );

    mul8_booth_wallace u_booth (
        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_booth)
    );

    initial begin
        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = -128; ib < 128; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_s8 = ib[7:0];
                #1;
                if (p_base !== p_booth) begin
                    $display("MISMATCH a=%0d b=%0d base=%0d booth=%0d", ia,
                        ↪ ib, p_base, p_booth);
                    $finish;
                end
            end
        end
        $display("mul8 vs booth/wallace: PASS");
        $finish;
    end
endmodule
```

## testbench tb\_simd\_mul\_compare 的 Verilog 源码

对应文件: ../sim/tb\_simd\_mul\_compare.v

```

`timescale 1ns/1ps

module tb_simd_mul_compare;
    reg [31:0] a_u8x4;
    reg [31:0] b_s8x4;
    wire [63:0] p_booth_s16x4;
    wire [63:0] p_dsp_s16x4;

    integer ia;
    integer ib;
    integer lane;
    integer expected;

    simd_mul8x4 u_booth (
        .a_u8x4(a_u8x4),
        .b_s8x4(b_s8x4),
        .p_s16x4(p_booth_s16x4)
    );

    simd_mul8x4_dsp u_dsp (
        .a_u8x4(a_u8x4),
        .b_s8x4(b_s8x4),
        .p_s16x4(p_dsp_s16x4)
    );

    task automatic check_lane_equal;
        input integer lane_idx;
        input integer exp_val;
        reg signed [15:0] booth_lane;
        reg signed [15:0] dsp_lane;
        begin
            booth_lane = p_booth_s16x4[lane_idx*16 +: 16];
            dsp_lane = p_dsp_s16x4[lane_idx*16 +: 16];
            if (booth_lane !== exp_val || dsp_lane !== exp_val || booth_lane
↪ !== dsp_lane) begin
                $display("tb_simd_mul_compare FAIL lane=%0d got_booth=%0d
↪ got_dsp=%0d exp=%0d",
                    lane_idx, booth_lane, dsp_lane, exp_val);
                $finish;
            end
        end
    endtask

    task automatic check_vector_pattern;
        input [31:0] a_vec;
        input [31:0] b_vec;

```

```

    reg signed [15:0] exp0;
    reg signed [15:0] exp1;
    reg signed [15:0] exp2;
    reg signed [15:0] exp3;
    begin
        a_u8x4 = a_vec;
        b_s8x4 = b_vec;
        exp0 = $signed({1'b0, a_vec[7:0]}) * $signed(b_vec[7:0]);
        exp1 = $signed({1'b0, a_vec[15:8]}) * $signed(b_vec[15:8]);
        exp2 = $signed({1'b0, a_vec[23:16]}) * $signed(b_vec[23:16]);
        exp3 = $signed({1'b0, a_vec[31:24]}) * $signed(b_vec[31:24]);
        #1;
        check_lane_equal(0, exp0);
        check_lane_equal(1, exp1);
        check_lane_equal(2, exp2);
        check_lane_equal(3, exp3);
    end
endtask

initial begin
    for (ia = 0; ia < 256; ia = ia + 1) begin
        for (ib = -128; ib < 128; ib = ib + 1) begin
            a_u8x4 = {ia[7:0], ia[7:0], ia[7:0], ia[7:0]};
            b_s8x4 = {ib[7:0], ib[7:0], ib[7:0], ib[7:0]};
            expected = ia * ib;
            #1;
            for (lane = 0; lane < 4; lane = lane + 1) begin
                check_lane_equal(lane, expected);
            end
        end
    end

    check_vector_pattern({8'd255, 8'd128, 8'd17, 8'd3},
        {8'sd127, -8'sd64, 8'sd5, -8'sd7});
    check_vector_pattern({8'd0, 8'd1, 8'd200, 8'd255},
        {-8'sd128, 8'sd0, -8'sd1, 8'sd1});
    check_vector_pattern({8'd12, 8'd34, 8'd56, 8'd78},
        {8'sd9, -8'sd10, 8'sd11, -8'sd12});

    $display("tb_simd_mul_compare PASS");
    $finish;
end
endmodule

```

## 2 里程碑 3：优化与 Vivado 性能验证

### 2.1 综合设置

综合脚本 `scripts/run_vivado_multi_synth.tcl` 会依次对以下 5 个顶层执行 `synth_design`:

- `cnn_compare_base_top`
- `cnn_compare_booth_top`
- `cnn_compare_booth_simd_top`
- `cnn_compare_booth_simd_pipe_top`
- `cnn_compare_booth_simd_pipe_dsp_top`

所有数据均来自 Vivado 2025.1 `utilization_report` 与 `timing_summary_report`。

由于本报告在 5ns 约束下比较不同 RTL 结构，因此将估算最小时钟周期写为

$$T_{\min} \approx 5.000 - \text{WNS}$$

再用

$$f_{\max} \approx \frac{1000}{T_{\min}}$$

估算最高工作频率，用于横向比较不同结构。

### 2.2 主四组资源与时序对比

主对比表仅统计前 4 组，不包含 DSP 替换版。结果如表 2 所示。

表 2: 主四组实验的资源与时序对比

| 实验组                       | LUT  | FF  | DSP | WNS/ns | 估算 $f_{\max}$ /MHz | 周期数 |
|---------------------------|------|-----|-----|--------|--------------------|-----|
| 串行                        | 550  | 304 | 0   | -5.546 | 94.82              | 96  |
| 串行 + BoothWallace         | 532  | 304 | 0   | -5.883 | 91.89              | 96  |
| BoothWallace + SIMD       | 8095 | 33  | 0   | -6.030 | 90.66              | 1   |
| BoothWallace + SIMD + 流水线 | 8097 | 287 | 0   | -1.625 | 150.94             | 3   |

结合表 2 与时序报告，可得到以下结论：

- 第 2 组在串行框架中将 LUT 从 550 降到 532，但 WNS 从 -5.546 ns 变为 -5.883 ns，说明当前关键路径仍主要受串行卷积累加与布线影响，而不是单纯由乘法器级数决定；
- 第 3 组将计算周期压缩到 1 个周期，但由于卷积、激活和池化几乎全部落在同一条组合路径上，输入端口到输出寄存器的路径最长，WNS 反而最差；
- 第 4 组加入真实三级流水后，WNS 明显改善到 -1.625 ns，估算最高频率提升到 150.94 MHz，是主四组中时钟能力最强的实现；

- 若按估算频率折算单次计算时延，则第 1/2/3/4 组约为 1012.44 ns、1044.73 ns、11.03 ns 和 19.88 ns。也就是说，第 3 组在当前结构下具有更短的单次结果延迟，而第 4 组则以更好的时钟能力和更规则的吞吐结构取胜。

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -5.546 ns   | Worst Hold Slack (WHS): -0.998 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -776.745 ns | Total Hold Slack (THS): -47.531 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 213        | Number of Failing Endpoints: 427   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 938          | Total Number of Endpoints: 938     | Total Number of Endpoints: 305                    |

Timing constraints are not met.

图 1: 第一组实验的时序报告

| Name                     | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded IOB<br>(400) | BUFGCTRL<br>(32) |
|--------------------------|----|------------------------|-----------------------------|---------------------|------------------|
| > N cnn_compare_base_top |    | 550                    | 304                         | 308                 | 1                |

图 2: 第一组实验的综合报告

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -5.883 ns   | Worst Hold Slack (WHS): -0.998 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -827.640 ns | Total Hold Slack (THS): -47.531 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 213        | Number of Failing Endpoints: 427   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 938          | Total Number of Endpoints: 938     | Total Number of Endpoints: 305                    |

Timing constraints are not met.

图 3: 第二组实验的时序报告

| Name                      | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | F7<br>Muxes | Bonded<br>IOB | BUFGCTRL<br>(32) |
|---------------------------|----|------------------------|-----------------------------|-------------|---------------|------------------|
| > N cnn_compare_booth_top |    | 532                    | 304                         | 2           | 308           | 1                |

图 4: 第二组实验的综合报告

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -6.030 ns   | Worst Hold Slack (WHS): -1.032 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -208.245 ns | Total Hold Slack (THS): -28.785 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 65         | Number of Failing Endpoints: 66    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 131          | Total Number of Endpoints: 131     | Total Number of Endpoints: 34                     |

Timing constraints are not met.

图 5: 第三组实验的时序报告

| Name                           | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded<br>IOB (400) | BUFG<br>CTRL<br>(32) |
|--------------------------------|----|------------------------|-----------------------------|---------------------|----------------------|
| > N cnn_compare_booth_simd_top |    | 8095                   | 33                          | 308                 | 1                    |

图 6: 第三组实验的综合报告

| Setup                                   | Hold                                | Pulse Width                                       |
|-----------------------------------------|-------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -1.625 ns   | Worst Hold Slack (WHS): -1.032 ns   | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -142.590 ns | Total Hold Slack (THS): -165.875 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 150        | Number of Failing Endpoints: 594    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 891          | Total Number of Endpoints: 891      | Total Number of Endpoints: 288                    |

Timing constraints are not met.

图 7: 第四组实验的时序报告

| Name                                | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded IOB<br>(400) | BUFGCTRL<br>(32) |
|-------------------------------------|----|------------------------|-----------------------------|---------------------|------------------|
| > N cnn_compare_booth_simd_pipe_top |    | 8097                   | 287                         | 308                 | 1                |

图 8: 第四组实验的综合报告

## 2.3 第 5 组: 在第 4 组基础上换用 DSP

第 5 组仅作为第 4 组的 DSP 替换对照, 不进入主四组主表。其比较结果如表 3 所示。

表 3: 第 4 组与第 5 组对比

| 指标                 | 第 4 组: BoothWallace + SIMD + 流水线 | 第 5 组: DSP 替换版 |
|--------------------|----------------------------------|----------------|
| LUT                | 8097                             | 1372           |
| FF                 | 287                              | 287            |
| DSP                | 0                                | 81             |
| WNS/ns             | -1.625                           | -2.054         |
| 估算 $f_{\max}$ /MHz | 150.94                           | 141.76         |
| 系统仿真周期数            | 3                                | 3              |

这一组实验的结论比较明确:

- 换用 DSP48E1 后, LUT 从 8097 大幅下降到 1372, 逻辑资源减少约 83.1%;
- 但在当前 200 MHz 约束下, DSP 版的 WNS 并没有改善, 反而从 -1.625 ns 变为 -2.054 ns;
- 功能与延迟周期保持不变, 说明第 5 组达成了“只改实现、不改功能”的目标;
- 因此, 对本工程当前 RTL 而言, DSP 替换带来的主要收益是逻辑资源迁移, 而不是直接的性能提升。

| Setup                                   | Hold                                | Pulse Width                                       |
|-----------------------------------------|-------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -2.054 ns   | Worst Hold Slack (WHS): -1.032 ns   | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -308.675 ns | Total Hold Slack (THS): -153.182 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 204        | Number of Failing Endpoints: 468    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 891          | Total Number of Endpoints: 891      | Total Number of Endpoints: 288                    |

Timing constraints are not met.

图 9: 第五组实验的时序报告

| Name                                    | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | DSPs<br>(740) | Bonded<br>IOB | BUFGCTRL<br>(32) |
|-----------------------------------------|----|------------------------|-----------------------------|---------------|---------------|------------------|
| > N cnn_compare_booth_simd_pipe_dsp_top |    | 1372                   | 287                         | 81            | 308           | 1                |

图 10: 第五组实验的综合报告

## 2.4 优化版本系统仿真结果

系统级仿真对 5 个顶层分别拉起 **start**，并显式检查输出与延迟行为。结果如表 4 所示。

表 4: 系统级仿真结果

| 实验组                       | 输出结果         | 延迟周期 |
|---------------------------|--------------|------|
| 串行                        | 32'h9087635A | 96   |
| 串行 + BoothWallace         | 32'h9087635A | 96   |
| BoothWallace + SIMD       | 32'h9087635A | 1    |
| BoothWallace + SIMD + 流水线 | 32'h9087635A | 3    |
| DSP 替换版                   | 32'h9087635A | 3    |

XSIM 日志中可以直接看到如下 PASS 信息：

```

CASE=1 PASS cycles=96 out=9087635a
CASE=2 PASS cycles=96 out=9087635a
CASE=3 PASS cycles=1 out=9087635a
CASE=4 PASS cycles=3 out=9087635a
CASE=5 PASS cycles=3 out=9087635a
ALL TESTS PASSED.

```

图 11 给出了第 4 组流水线版本的关键内部波形。可以看到 **vld\_pipe** 按 001 → 010 → 100 依次推进，卷积结果寄存、ReLU 结果寄存和池化输出寄存依次有效，最终在第 3 个周期输出 0x9087635A。

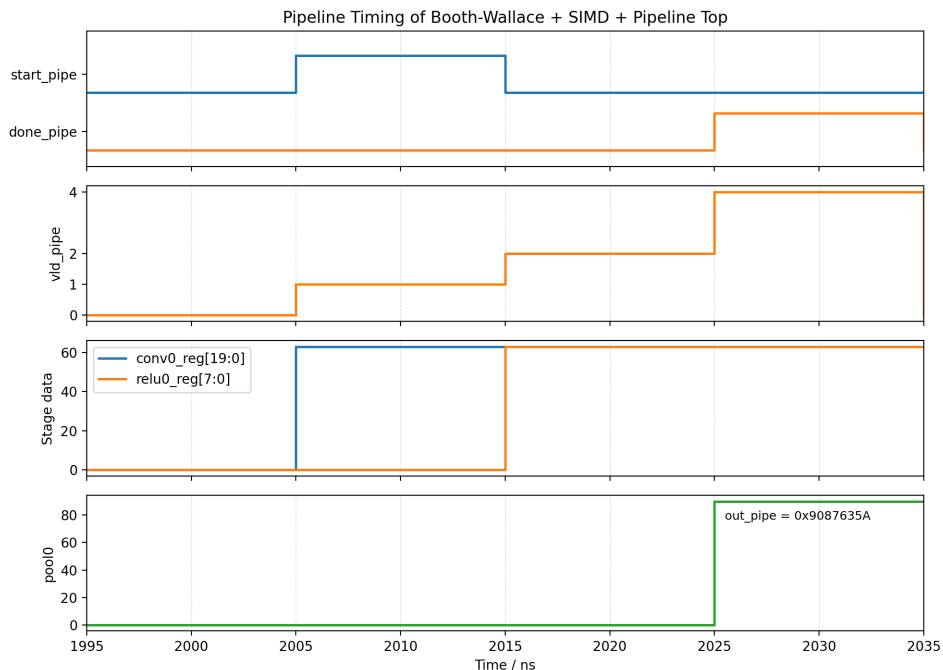


图 11: 流水线波形: 卷积寄存、ReLU 寄存与输出寄存的三级推进

### 3 实验收获

- 完整实现并验证了 8 位定点卷积、ReLU 激活和平均池化三段链路, 对 CNN 基础算子在 FPGA 上的组织方式有了更具体的理解。
- 通过把 5 组实验拆成独立顶层, 进一步体会到“便于综合分析的结构”与“便于功能复用的结构”并不总是同一件事。
- BoothWallace 乘法器、SIMD 路径和 3 级流水线都已经在工程中落地, 并通过统一 testbench 验证了功能一致性。
- 从 Vivado 报告中可以看出, 优化并不总会直接转化为更高主频; 关键路径位置、寄存器边界和布线代价同样会改变最终结果。
- DSP 替换版说明了另一类典型工程取舍: 它显著降低了 LUT 占用, 但在当前结构下并未直接带来性能收益。

### 4 附: 源码目录

- project1/src/baseline/: 串行基线实现, 包括 mul8.v、add8.v、relu\_s20.v、div4.v、cnn\_chain\_core\_base.v 等;
- project1/src/optimized/: BoothWallace、SIMD、流水线和 DSP 版本实现;
- project1/sim/: 模块级与系统级 testbench;
- project1/scripts/: Vivado 批处理与波形导出脚本;



- `project1/report/`: 本报告与图片素材;
- `project1/build/`: 综合输出与仿真波形。